第 12 章：用 Requests 调用 HTTP API，并守住 JSON 边界

一句话先说透：**调用 HTTP API，不是“发请求后把 JSON 转成字典”这么简单，而是要依次确认传输、HTTP 状态、JSON 语法、数据结构和业务结果都成功。**

如果你熟悉 Spring MVC、`RestClient`、`WebClient`、Jackson 和 DTO 校验，可以先把 Python 的 `requests` 理解成一个轻量的同步 HTTP 客户端。它能帮你发送请求、管理连接和解析 JSON，但不会像完整的 Spring 客户端封装那样，自动替你完成 DTO 映射、Bean Validation、统一异常转换和重试治理。

学完本章，你应该能够：

- 用 `params=`、`json=` 和 `data=` 把数据放到正确的位置；
- 分清 Python `dict`、JSON 文本 `str` 和网络字节 `bytes`；
- 正确判断 HTTP 成功、JSON 可解析、字段合法和业务成功；
- 给请求设置合理的连接超时与读取超时；
- 使用 `Session` 复用连接，并明确资源由谁关闭；
- 把网络、状态码、协议数据和业务错误分开暴露。

本章只讨论同步 Requests。异步 HTTP、Pydantic、自动重试和总 deadline 会在需要这些能力的章节再展开。

一、先建立 Java → Python 心智模型

Java 中熟悉的概念	Python / Requests 中的对应物	必须注意的不等价之处
`java.net.http.HttpClient`、Spring `RestClient`	`requests.Session`	都能发送同步请求并复用连接；Requests 的默认抽象更轻，不自带 Spring 式拦截器和统一 DTO 体系
`UriComponentsBuilder.queryParam(...)`	`params={...}`	都负责查询参数编码，不要手工拼接中文、空格和 `&`
Jackson `ObjectMapper.writeValueAsString`	`json.dumps(...)`	都把内存对象序列化成 JSON 文本；`requests` 的 `json=` 还能代做这一步

t72502x6dg8fpjj0sff40000gp/T/crossnote-f-6a8341d31cd7ee3e1dd0c920a48.html

eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单

Jackson `readValue(...)`	`response.json()` 或 `json.loads(...)`	Python 默认得到 `dict` 、 `list` 等通用对象，不会自动变成强类型 DTO
`ResponseEntity<T>`	`requests.Response`	`Response` 同时提供状态、头、字节、文本和 JSON 入口，但 `T` 不会自动校验
DTO / Java `record`	`@dataclass`	`dataclass` 主要减少样板代码，类型注解默认不会在运行时自动校验外部数据
`AutoCloseable` + try-with-resources	`with requests.Session() as session`	都表达资源生命周期；退出 `with` 时 Session 会关闭连接池资源
`throw new XxxException(..., cause)`	`raise XxxError(...) from error`	都能在增加业务语义的同时保留底层异常原因

最重要的差异是：Java 项目常把 HTTP 客户端、Jackson、DTO 和校验框架组合成一层完整的客户端 SDK；原生 Requests 只提供较底层的构件。**边界是否可信，仍然要由你的应用代码判断。**

二、一次 API 调用到底经过哪些边界

把一次调用拆成下面五层，排错时就不会只剩一句“接口挂了”：

```
构造请求 → 建立连接并传输 → 检查 HTTP 状态 → 解析 JSON → 校验结构与业务结果
```

层次	要回答的问题	常见失败
请求构造	方法、URL、参数、请求头和 body 是否符合接口合同?	URL 拼错、查询词为空、凭据缺失、body 编码错误
HTTP 传输	是否成功连接，并收到服务端响应?	连接被拒绝、DNS/TLS 问题、连接或读取超时
HTTP 状态	服务端是否在 HTTP 层接受了请求?	401、404、429、500，或者合同不接受的 204/3xx
JSON 语法	响应正文是否为合法 JSON?	网关返回 HTML、正文截断、JSON 格式错误

2502x6dg8fpjj0sff40000gp/T/crossnote-f-6a8341d31cd7ee3e1dd0c920a48.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计

JSON 语法	响应正文是否为合法 JSON?	网关返回 HTML、正文截断、JSON 格式错误
数据与业务	字段、类型、空值和业务码是否符合合同?	缺字段、`null`、类型漂移、`code != "OK"`

这五层不能互相替代。例如：

- 404 的正文完全可能是合法 JSON；
- HTTP 200 的正文可能是一段 HTML；
- HTTP 200 加合法 JSON，也可能包含 `{"code": "QUOTA_EXCEEDED"}`；
- 字段存在但值为 `null`，与服务端根本没发送这个字段，不是同一种情况。

这和 Java 服务中的分层非常像：TCP 成功不等于 Controller 成功，Controller 返回 200 不等于 Jackson 一定能反序列化，反序列化成功也不等于业务状态有效。

三、请求由方法、地址、参数、头和 body 组成

1. GET 查询：用 `params=`，不要手拼 URL

```
import requests

response = requests.get(
    "https://api.example.com/documents",
    params={"query": "Agent 入门", "page": 1},
    headers={"Accept": "application/json"},
    timeout=(3.05, 20.0),
)
```

`params` 接收 Python 字典。Requests 会把它编码到 URL 的 query string 中，因此中文、空格、`&` 和 `=` 都会被正确处理。

可以把它类比为 Spring 的 URI Builder：你提供参数值，客户端负责 percent-encoding。不要写下面这种代码：

```
# 反例：query 中只要出现空格、中文或 &，就可能生成错误 URL。
url = f"https://api.example.com/documents?query={query}"
```

查询参数经常进入访问日志、代理日志和监控系统，所以 token 不应放在 query string 中。认证信息通常放到 `Authorization` 请求头：

```
headers = {"Authorization": f"Bearer {token}"}
```

2. POST body：先看服务端要求哪种编码

写法	数据位置	Requests 会做什么	典型场景
`params={"q": "Python"}`	URL 查询串	URL 编码	筛选、分页、搜索
`json={"q": "Python"}`	body	JSON 序列化，并设置 JSON `Content-Type`	REST/JSON API
`data={"q": "Python"}`	body	表单编码	`application/x-www-form-urlencoded` 接口
`data=json_text`	body	原样发送 `str` 或 `bytes`	自己已经完成精确编码、签名等特殊协议

普通 JSON API 的正确写法是：

```
payload = {"query": "Agent 入门", "limit": 2}

response = requests.post(
    "https://api.example.com/search",
    json=payload,
    timeout=(3.05, 20.0),
)
```

这里传给 `json=` 的应该是 **Python 对象**。Requests 会完成 JSON 序列化和请求头设置。

下面是 Java 工程师很容易写出的重复序列化反例：

```
import json

json_text = json.dumps({"query": "Agent"})
requests.post(url, json=json_text)  # 错：json_text 会被当作普通字符串再编码一次
```

服务端解析一次后得到的是字符串，而不是对象。它相当于先用 Jackson 得到 JSON 字符串，又把这个字符串交给 Jackson 当普通值再序列化一次。

选择方法可以压缩成三句话：

1. 手里是 Python 对象，服务端要 JSON：使用 `json=python_value` 。
2. 服务端明确要表单：使用 `data=dict_value` 。
3. 协议要求自己控制原始正文：使用 `data=str_or_bytes` ，同时自行设置正确的 `Content-Type` 。

四、`dict` 、JSON `str` 和 `bytes` 是三个不同阶段

```
import json

python_value = {"query": "Agent 入门"}                    # dict
json_text = json.dumps(python_value, ensure_ascii=False)   # str
body_bytes = json_text.encode("utf-8")                     # bytes
```

数据流是：

```
Python dict/list
    | json.dumps
    ▼
JSON 文本 str
    | encode("utf-8")
    ▼
可传输的 bytes
```

反方向则是：

```
响应 bytes
    | 按字符编码解码
    ▼
JSON 文本 str
    | json.loads / response.json()
    ▼
Python dict/list/str/int/bool/None
```

可以类比 Java 中的 `Map<String, Object>` 、JSON `String` 和 UTF-8 `byte[]` 。外观看起来像 JSON，不代表类型已经是可按字段访问的对象。

几个常见判断：

vf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-6a8341d31cd7ee3e1dd0c920a48.html

目关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下l

几个常见判断：

- `dict` 可以用 `value["query"]` 取字段，但它还不是 JSON 文本；
- `str` 是字符序列，必须先 `json.loads(...)` 才能得到 JSON 对应的 Python 值；
- `bytes` 是字节序列，必须知道字符编码才能安全变成文本；
- JSON 顶层可以是数组、字符串、数字、布尔值或 `null`，所以 `response.json()` 不保证返回 `dict`。

如果输出里突然出现很多 `\"`，先检查 `type(value)`。这通常表示字符串被重复 JSON 序列化了。

五、正确读取响应：先看状态，再解析，再校验

一个 `Response` 有几种常用视图，它们回答的问题不同：

写法	得到什么	适合做什么
`response.status_code`	`int` 状态码	按接口合同判断 200、201、204 等
`response.raise_for_status()`	4xx/5xx 时抛 `HTTPError`	快速拒绝 HTTP 错误
`response.content`	`bytes`	文件、图片或需要自行解码的内容
`response.text`	`str`	普通文本或受控的错误正文预览
`response.json()`	JSON 对应的 Python 值	解析 JSON；之后仍需校验结构

一个可靠的 JSON API 处理顺序通常是：

```
response.raise_for_status()
if response.status_code != 200:  # 本接口合同只接受 200
    raise RuntimeError(f"预期 200，实际 {response.status_code}")

try:
    payload = response.json()
except requests.JSONDecodeError as error:
    raise RuntimeError("响应不是合法 JSON") from error

if not isinstance(payload, dict):
    raise TypeError("响应 JSON 顶层必须是 object")
```

`raise_for_status()` 主要拒绝 4xx 和 5xx。它不负责以下事情：

`raise_for_status()` 主要拒绝 4xx 和 5xx。它不负责以下事情：

- 不保证状态恰好是接口合同要求的 200；
- 不保证正文是 JSON；
- 不保证 JSON 顶层是对象；
- 不保证字段存在、类型正确；
- 不保证业务码成功。

缺字段和 `null` 要分开

JSON 的 `null` 会被解析成 Python 的 `None`：

```
if "title" not in document:
    raise ValueError("缺少 title")
if document["title"] is None:
    raise ValueError("title 不能为 null")
```

不要只写 `document.get("title") is None`，因为它会把“缺少字段”和“明确传了 null”合并。是否允许省略或允许 `null`，应由接口合同决定。

类型注解不是运行时校验器

即使函数标注返回 `Document`，网络返回的数据也不会自动变成 `Document`。通常需要先验证原始 `dict`，再构造稳定对象：

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Document:
    id: int
    title: str
    content: str
```

`@dataclass` 可以类比 Java `record` 或由 Lombok 生成的 DTO，但不能完全等同：它会生成构造器、比较和展示等样板方法，**默认不会检查 `id` 运行时是否真的是 `int`**。本章示例因此在构造 `Document` 前显式检查外部数据。

另一个 Python 特有边界是 `bool` 属于 `int` 的子类：`isinstance(True, int)` 为真。如果协议要求“真正的整数 ID”，可使用 `type(value) is int` 排除布尔值。

六、timeout 限制的是网络等待，不是整个业务耗时

Requests 默认不会自动给请求设置 timeout。生产代码省略 `timeout=`，可能让工作线程长时间等待。

```
response = session.get(url, timeout=(3.05, 20.0))
#                                    |     └ read timeout
#                                    └──────── connect timeout
```

- **connect timeout**：等待建立连接的上限；
- **read timeout**：连接建立后，等待响应字节的 socket 空闲上限。

`timeout=(3.05, 20.0)` 不表示整个函数一定在 23.05 秒内结束。域名可能解析出多个地址并依次尝试；只要响应持续在 read timeout 内送来新字节，大文件下载也可能远超 20 秒；JSON 解析和后续业务处理更不在这个计时范围内。

可以把它类比为 Java HTTP 客户端的连接/读取超时，但不能把它当成 `CompletableFuture.orTimeout(...)` 或上层请求 deadline。若业务需要一个覆盖多次调用、解析和后续处理的总截止时间，必须在更高层单独设计。

七、多次请求使用 `Session`，并明确所有权

一次独立调用可以使用模块函数：

```
response = requests.get(url, timeout=(3.05, 20.0))
```

同一服务有多次请求时，更适合使用 `Session`：

```
with requests.Session() as session:
    session.headers.update({"X-Client": "document-service"})
    first = session.get(first_url, timeout=(3.05, 20.0))
    second = session.get(second_url, timeout=(3.05, 20.0))
```

Session 的作用包括：

- 通过底层连接池复用连接，减少重复建连；
- 在多次请求之间保留 cookie；
- 保存公共 headers、认证等默认配置。

它可以类比为复用的 Java `HttpClient`，而 `with` 可以类比 try-with-resources。连接池允许连接复用，但服务端、代

它可以类比为复用的 Java `HttpClient`，而 `with` 可以类比 try-with-resources。连接池允许连接复用，但服务端、代理或响应头仍可能关闭连接，所以不能假定所有请求永远使用同一个 socket。

如果函数接受调用者传入的 Session，推荐遵守“谁创建，谁关闭”：

- 调用者传入 Session：函数只借用，不关闭；
- 函数内部创建 Session：函数必须在正常返回和异常路径都关闭它。

八、错误要按层分类，并保留 cause

不要这样处理所有失败：

```
# 反例：404、连接失败、非法 JSON 和真实空结果都被伪装成同一个值。
try:
    return response.json()
except Exception:
    return {}
```

更可靠的客户端会区分：

失败层	Requests / 应用中的例子	调用方通常怎么处理
配置与输入	缺服务地址、空查询词	修正配置或立即拒绝调用
传输	`ConnectionError`、`Timeout`	检查网络、服务状态和超时预算
HTTP 状态	`HTTPError`，或应用的 `HttpStatusFailure`	按 401/404/429/5xx 制定策略
JSON 语法	`JSONDecodeError`	记录受控正文预览，检查网关或服务输出
数据结构	缺字段、`null`、类型错误	视为协议不兼容，不能假装没有结果
业务结果	`code != "OK"`	按稳定业务码处理配额、状态或用户提示

转换底层异常时使用异常链：

```
try:
    response = session.get(url, timeout=(3.05, 20.0))
except requests.Timeout as error:
    raise RequestTimeout("文档服务请求超时") from error
except requests.ConnectionError as error:
```

71z72502x6dg8fpjj0sff40000gp/T/crossnote-f-6a8341d31cd7ee3e1dd0c920a48.html

关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下4

转换底层异常时使用异常链：

```
try:
    response = session.get(url, timeout=(3.05, 20.0))
except requests.Timeout as error:
    raise RequestTimeout("文档服务请求超时") from error
except requests.ConnectionError as error:
    raise ConnectionFailure("无法连接文档服务") from error
```

`raise ... from error` 类似 Java 构造异常时传入 `cause`：上层看到的是稳定的应用异常，排障时仍能沿 `__cause__` 找到底层 Requests/urllib3 原因。

捕获顺序也有意义。Requests 的 `ConnectTimeout` 同时属于 `Timeout` 和 `ConnectionError`，因此本章先捕获 `requests.Timeout`，再捕获 `requests.ConnectionError`，避免把连接超时归入普通连接失败。

至于重试，不能看到网络异常就无脑重发。GET 通常设计为幂等，但 POST 可能创建订单、扣费或发消息。只有确认方法语义、幂等键、最大次数、退避和总时间预算后，才应加入自动重试。

九、一个可维护的 API 函数应该承诺什么

本章项目提供 `fetch_document(query)`，它体现的是一份客户端合同，而不是需要先背下来的代码流程：

- 输入 `query` 必须是非空字符串，请求发出前先校验；
- `base_url` 和 token 可以显式传入，未传时才从环境变量读取；
- 查询参数交给 `params=` 编码，认证信息放请求头；
- 调用方可以传入共享 Session，也可以让函数管理私有 Session；
- 只有 HTTP、JSON、结构和业务规则全部通过，才返回 `Document`；
- 每一层失败都有不同异常，不用空字典伪装成功；
- 转换异常时保留底层 cause；
- 诊断日志中的 token 必须脱敏。

显式参数优先，最利于单元测试：

```
document = fetch_document(
    "Agent",
    base_url="https://api.example.com",
    api_token=token_from_secret_store,
)
```

z72502x6dg8fpjj0sff40000gp/T/crossnote-f-6a8341d31cd7ee3e1dd0c920a48.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计

应用入口也可以通过环境变量注入：

```
export DOCUMENT_API_BASE_URL='https://api.example.com'
export DOCUMENT_API_TOKEN='由部署平台注入的真实值'
```

不要把真实 token 写进源码、README、完整 URL 或异常消息。日志中除了凭据，查询词、响应正文、prompt 和模型输出也可能含敏感数据，应按数据等级决定是否记录。

十、运行本章的本地实验

1. 环境与依赖

仓库根目录的 `pyproject.toml` 要求 Python `>=3.14`。本章直接依赖记录在 `requirements.txt`：

```
requests==2.34.2
```

在仓库根目录执行：

```
.venv/bin/python -V
.venv/bin/python -m pip install -r java_to_python/chapters/12_http_json/requirements.txt
.venv/bin/python -m pip check
.venv/bin/python -B java_to_python/chapters/12_http_json/example.py
```

`pip install` 会修改当前 `.venv` 的依赖；请确认它就是你准备用于本仓库的虚拟环境。`-B` 只表示运行时不生成 `.pyc` 文件。

当前 `java_to_python/run_all.py` 只登记了前 10 章，所以本章要用上面的独立命令运行，不能使用 `--chapter 12`。

2. 本章文件分别负责什么

文件	作用
`README.md`	建立 HTTP、JSON、Session、timeout 和异常边界的心智模型
`example.py`	可运行的 Requests 客户端与 `fetch_document()`
`local_http_fixture.py`	自动启动和关闭本机 HTTP 服务，稳定制造成功与失败响应
`requirements.txt`	本章直接运行依赖

:72502x6dg8fpjj0sff40000gp/T/crossnote-f-6a8341d31cd7ee3e1dd0c920a48.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计费

默认入口只访问 `127.0.0.1` 上临时启动的服务，不访问公网，也不需要真实 API Key。fixture 使用随机空闲端口，退出上下文后会关闭监听 socket 和服务线程。

3. 运行时重点观察什么

不需要先逐行阅读代码。先运行并寻找这些现象：

1. 同一个值在 `dict`、JSON `str` 和 UTF-8 `bytes` 三种形态下类型不同；
2. `params=` 进入 URL，`json=dict` 生成 JSON body，`data=dict` 生成表单；
3. 404 正文虽然能解析成 JSON，`raise_for_status()` 仍会拒绝它；
4. 两次查询通过同一个 Session 复用本地连接；
5. 连接失败、read timeout、HTTP 错误、非法 JSON、缺字段、`null`、错类型和业务失败落入不同异常；
6. 最后 Session、监听端口和服务线程都被关闭。

慢响应场景是故意触发的受控 `ReadTimeout`。客户端超时后，fixture 会释放等待事件并完成清理，所以这条异常输出属于实验结果。

十一、Java 工程师最容易踩的坑

1. **把 `response.json()` 当成 DTO 反序列化加校验。** 它只负责 JSON 语法解析，返回值甚至不一定是字典。
2. **先 `json.dumps`，又把结果传给 `json=`。** 这会发生重复序列化，服务端得到字符串而不是对象。
3. **用 `data=dict` 发送 JSON。** Requests 会按表单编码；JSON API 应使用 `json=dict`。
4. **只依赖 `raise_for_status()` 判断成功。** 它不检查精确成功状态、JSON、字段或业务码。
5. **认为类型注解会在运行时拦截脏数据。** Python 注解主要服务于阅读和静态检查，外部输入仍需显式校验或校验库。
6. **省略 timeout。** Requests 没有默认请求超时，线程可能长时间等待。
7. **把 `(connect, read)` 当成总超时。** 它不是覆盖整个业务流程的 deadline。
8. **每次请求都新建客户端。** 同一服务的连续调用应考虑复用 Session 和连接池。
9. **捕获 `Exception` 后返回 `{}`。** 这会把故障伪装成正常空结果，并丢失排障证据。
10. **把 token 和完整响应写进日志。** HTTP 客户端日志往往会进入长期保存的集中系统。

十二、练习

练习 1：验证四种发送方式

把同一个 `{"query": "Agent", "limit": 2}` 分别传给：

- `json=dict_value`

十二、练习

练习 1：验证四种发送方式

把同一个 `{"query": "Agent", "limit": 2}` 分别传给：

- `json=dict_value`
- `data=dict_value`
- `data=json.dumps(dict_value)`
- `json=json.dumps(dict_value)`

记录每次请求的 `Content-Type`、原始 body 和服务端解析后的类型。目标是能亲眼区分 JSON 对象、表单、原始 JSON 文本和重复编码后的 JSON 字符串。

练习 2：增加顶层数组响应

让本地 fixture 返回 HTTP 200 和 JSON 数组。保持 `fetch_document()` 的返回合同不变，使它在数据校验层抛出明确异常，而不是泄漏 `TypeError`、`KeyError` 或返回空对象。

练习 3：为 204 定义合同

给 fixture 增加 HTTP 204 响应。先决定“查询文档”是否允许 204：

- 不允许：在 HTTP 状态层明确拒绝；
- 允许：设计一个能明确表达“无正文”的返回类型。

不要直接调用 `.json()`，再把解码失败解释成正常空结果。

本章小结

以后看到一个 API 调用，只要按下面顺序提问即可：

1. 请求是否构造正确？
2. 连接和传输是否完成？
3. HTTP 状态是否符合接口合同？
4. 正文是否为合法 JSON？
5. JSON 的结构、字段类型和业务码是否合格？

/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-6a8341d31cd7ee3e1dd0c920a48.html

需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下

- `data=dict_value`
- `data=json.dumps(dict_value)`
- `json=json.dumps(dict_value)`

记录每次请求的 `Content-Type`、原始 body 和服务端解析后的类型。目标是能亲眼区分 JSON 对象、表单、原始 JSON 文本和重复编码后的 JSON 字符串。

练习 2：增加顶层数组响应

让本地 fixture 返回 HTTP 200 和 JSON 数组。保持 `fetch_document()` 的返回合同不变，使它在数据校验层抛出明确异常，而不是泄漏 `TypeError`、`KeyError` 或返回空对象。

练习 3：为 204 定义合同

给 fixture 增加 HTTP 204 响应。先决定"查询文档"是否允许 204：

- 不允许：在 HTTP 状态层明确拒绝；
- 允许：设计一个能明确表达"无正文"的返回类型。

不要直接调用 `.json()`，再把解码失败解释成正常空结果。

本章小结

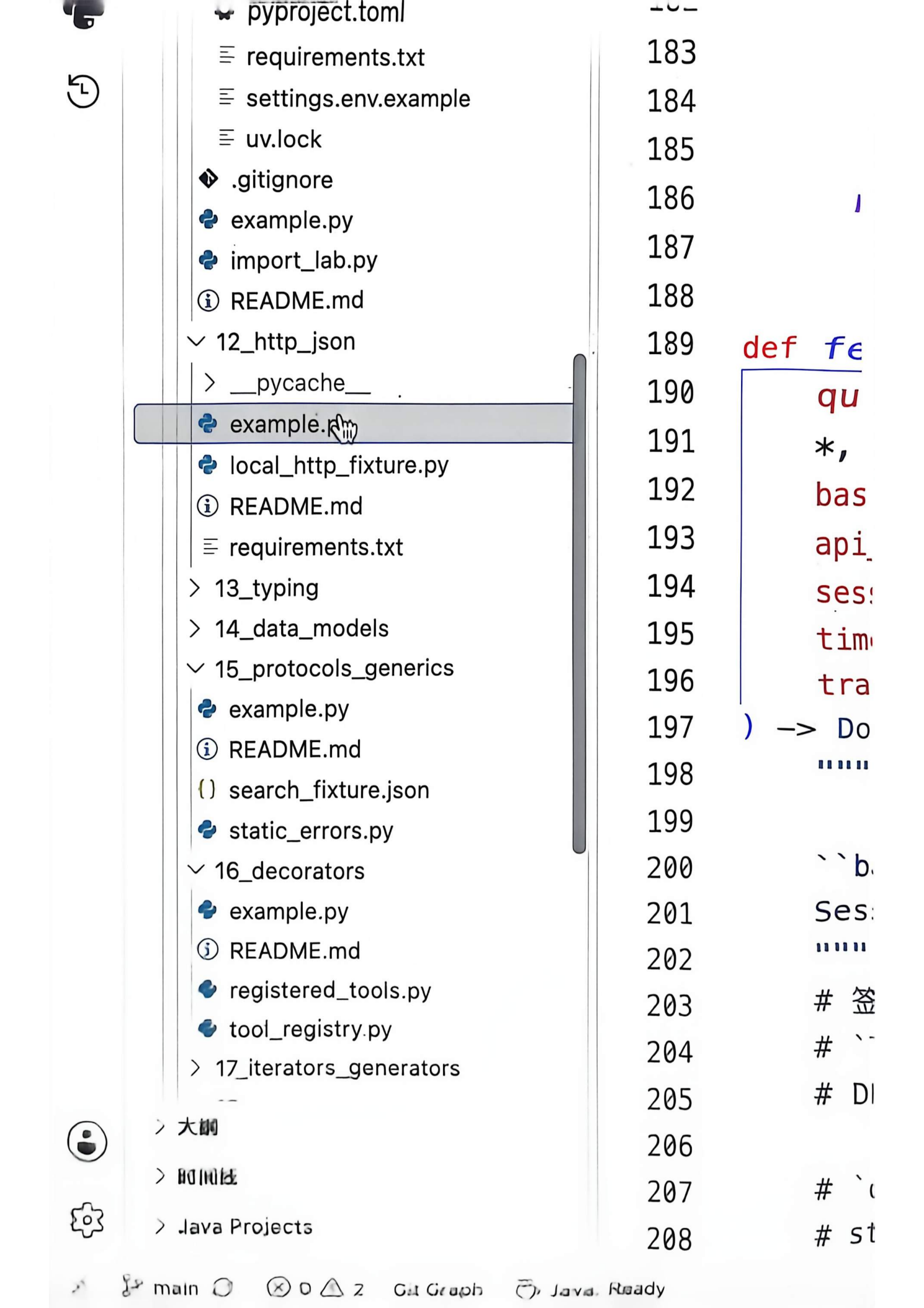
以后看到一个 API 调用，只要按下面顺序提问即可：

1. 请求是否构造正确?
2. 连接和传输是否完成?
3. HTTP 状态是否符合接口合同?
4. 正文是否为合法 JSON?
5. JSON 的结构、字段类型和业务码是否合格?

Requests 的 API 不多，真正重要的是建立这套边界意识。它会直接复用到后续数据库客户端、LLM API、MCP 工具调用和微服务集成中。

延伸阅读：

- Requests Quickstart
- Requests Advanced Usage：Session 与 timeout
- Requests API 与异常参考


pyproject.toml
requirements.txt
settings.env.example
uv.lock
.gitignore
example.py
import_lab.py
README.md
12_http_json
__pycache__
example.py
local_http_fixture.py
README.md
requirements.txt
13_typing
14_data_models
15_protocols_generics
example.py
README.md
search_fixture.json
static_errors.py
16_decorators
example.py
README.md
registered_tools.py
tool_registry.py
17_iterators_generators
大纲
时间线
Java Projects
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
def fe
qu
*,
bas
api_
ses
tim
tra
) -> Do
"""
``b
Ses
"""
签
`
DI
`
st
main
0
2
Git Graph
Java: Ready

```python
"""第 12 章：使用 requests 调用本地 API，并逐层处理 HTTP 与 JSON 边界。"""

# 建议按 main() 中的调用顺序阅读本文件：先辨认 Python 值、JSON 文本和 bytes，再比较
# params=/json=/data=，最后跟着 fetch_document() 走完请求、响应和失败分类。
# 默认入口只连接 local_http_fixture.py 启动的 127.0.0.1 服务，不会访问公网。

# 标准库中，json 负责 Python 对象与 JSON 文本之间的转换，os 读取进程环境变量；
# Callable 描述“可调用对象”的类型，dataclass 减少数据类样板代码，metadata 读取发行包版本。
# requests 是真正发出 HTTP 请求的第三方客户端；本地 fixture 则提供可自动关闭的教学服务器，
# 以及一个用于稳定制造“拒绝连接”的临时地址。
import json
import os
from collections.abc import Callable
from dataclasses import asdict, dataclass
from importlib import metadata

import requests
from local_http_fixture import LocalHttpFixture, refusing_connection_url

# 这两个值都是二元 tuple，依次表示建立连接的等待上限与相邻响应字节的等待上限。
# 它们不是“整个业务最多运行多久”；解析 JSON、校验字段和多次请求都不在总计时器内。
DEFAULT_TIMEOUT = (0.5, 0.3)
SLOW_RESPONSE_TIMEOUT = (0.5, 0.08)
# 这是类型别名：trace 回调接收阶段名 str 和任意 object，不返回有用结果。
# 它只服务于类型检查与阅读，Python 运行时不会像 Java 接口那样自动验证实现签名。
TraceCallback = Callable[[str, object], None]


# @dataclass 会在类定义时参与处理，自动生成 __init__、__repr__、__eq__ 等方法，
# 可类比减少 Java DTO/POJO 的样板代码；frozen=True 还会阻止字段在构造后被重新赋值。
@dataclass(frozen=True)
class Document:
    """通过所有边界校验后，调用者才能得到这个稳定的返回类型。"""

    id: int
    title: str
    content: str
```

```python
# @dataclass 会在类定义时参与处理，自动生成 __init__、__repr__、__eq__ 等方法，
# 可类比减少 Java DTO/POJO 的样板代码；frozen=True 还会阻止字段在构造后被重新赋值。
@dataclass(frozen=True)
class Document:
    """通过所有边界校验后，调用者才能得到这个稳定的返回类型。"""

    id: int
    title: str
    content: str


# 下面建立一棵应用异常继承树。调用者既可以捕获 DocumentApiError 统一处理所有预期失败，
# 也可以捕获某个子类，精确区分配置、传输、HTTP、JSON、数据结构和业务结果。
class DocumentApiError(RuntimeError):
    """文档 API 的预期失败基类；layer 明确指出失败发生在哪一层。"""

    # layer 是类属性，所有实例默认共享；子类只需覆盖这个标签，不必重复初始化逻辑。
    layer: str = "未知层"


class ConfigurationError(DocumentApiError):
    layer: str = "调用配置层"


class TransportError(DocumentApiError):
    layer: str = "HTTP 传输层"


class ConnectionFailure(TransportError):
    """DNS、拒绝连接、连接被重置等连接问题。"""


class RequestTimeout(TransportError):
    """connect timeout 或 read timeout。"""
```

```python

class HttpStatusFailure(DocumentApiError):
    layer: str = "HTTP 状态层"

    # 类型提示说明参数意图，但不负责运行时强制校验。先保存状态码，再把 message 交给
    # RuntimeError 的初始化链，这样 str(error) 和 traceback 都能显示错误消息。
    def __init__(self, status_code: int, message: str) -> None:
        self.status_code: int = status_code
        super().__init__(message)


class ResponseJsonFailure(DocumentApiError):
    layer: str = "JSON 语法层"


class ResponseValidationFailure(DocumentApiError):
    layer: str = "响应数据校验层"


class BusinessFailure(DocumentApiError):
    layer: str = "业务结果层"

    # 业务码作为结构化属性单独保留；调用者无需从自然语言错误消息中反向解析 code。
    def __init__(self, code: str, message: str) -> None:
        self.code: str = code
        super().__init__(message)


def print_section(title: str) -> None:
    """让终端输出与 README 小节一一对应。"""
    # f-string 会先计算花括号中的 title；开头的 \n 让不同实验之间空一行。
    print(f"\n=== {title} ===")


def mask_secret(secret: str | None) -> str:
    """日志只显示凭据是否存在及极少首尾字符，不输出完整值。"""
    # str | None 相当于 Java 语境下“可能没有值”，但这里只是类型提示
```

```python

def mask_secret(secret: str | None) -> str:
    """日志只显示凭据是否存在及极少首尾字符，不输出完整值。"""
    # str | None 相当于 Java 语境下“可能没有值”，但这里只是类型提示。
    # 布尔上下文会把 None 和空字符串都视为 False，因此二者都显示成未提供。
    if not secret:
        return "（未提供）"
    # 短凭据完全替换成同长度星号；字符串乘法会重复 "*"，不会修改原字符串。
    if len(secret) <= 4:
        return "*" * len(secret)
    # [前:后] 是切片：[:2] 取前两个字符，[-2:] 从倒数第二个字符取到末尾。
    return f"{secret[:2]}***{secret[-2:]}"


def trace_to_terminal(stage: str, value: object) -> None:
    """示例专用观察器；真实应用通常改用结构化日志并进一步限制响应正文。"""
    # object 表示入口可以接收任意 Python 对象。若已经校验成 Document，就用 asdict()
    # 新建普通 dict 供 JSON 序列化；它不会把原 Document 原地改成字典。
    if isinstance(value, Document):
        visible_value: object = asdict(value)
    else:
        visible_value = value
    # ensure_ascii=False 让中文保持可读；sort_keys=True 让字典键顺序稳定。
    # default=repr 表示遇到 json 不认识的对象时，退回它的 repr(...) 字符串用于教学输出。
    rendered: str = json.dumps(visible_value, ensure_ascii=False, sort_keys=True, default=repr)
    print(f"  {stage}: {rendered}")


def require_non_null_field(mapping: dict[str, object], field_name: str, *, location: str) -> object:
    """把“缺字段”和“字段明确为 null”分成两个可诊断错误。"""
    # 签名中的 * 使 location 成为仅限关键字参数，调用处必须写 location=...，
    # 读代码时能看出它是错误路径说明，而不是另一个待查询字段。
    # `in` 检查键是否存在，不能用 mapping.get(...) 一步代替，否则会把缺键与 JSON null
    #  （解析后是 None）折叠成同一种情况。
    if field_name not in mapping:
        # f-string 中的 !r 使用 repr(field_name)，所以错误里的字段名会带引号。
        raise ResponseValidationFailure(f"{location} 缺少字段 {field_name!r}")
```

```python

def require_non_null_field(mapping: dict[str, object], field_name: str, *, location: str) -
    """把“缺字段”和“字段明确为 null”分成两个可诊断错误。"""
    # 签名中的 * 使 location 成为仅限关键字参数，调用处必须写 location=...，
    # 读代码时能看出它是错误路径说明，而不是另一个待查询字段。
    # `in` 检查键是否存在，不能用 mapping.get(...) 一步代替，否则会把缺键与 JSON null
    #  （解析后是 None）折叠成同一种情况。
    if field_name not in mapping:
        # f-string 中的 !r 使用 repr(field_name)，所以错误里的字段名会带引号。
        raise ResponseValidationFailure(f"{location} 缺少字段 {field_name!r}")
    value: object = mapping[field_name]
    # None 是单例，按身份用 is 判断；不要用 == 套用 Java 的 null 比较经验。
    if value is None:
        raise ResponseValidationFailure(f"{location}.{field_name} 不能为 null")
    return value


def validate_document_payload(payload: object) -> Document:
    """校验 JSON 对应的 Python 对象，并只返回调用者真正需要的数据。"""
    # response.json() 的返回类型取决于 JSON 顶层，可能是 dict、list、str、数字或 None。
    # 因而参数先声明成最宽的 object，再在每一层用 isinstance 缩小为本 API 约定的结构。
    if not isinstance(payload, dict):
        raise ResponseValidationFailure("响应 JSON 顶层必须是 object/dict")

    # 先判断业务码。require_non_null_field 已保证 code 存在且不是 None，下一步再验证类型；
    # “存在”“非 null”“类型正确”和“值表示成功”是四条不同规则。
    code: object = require_non_null_field(payload, "code", location="响应")
    if not isinstance(code, str):
        raise ResponseValidationFailure("响应.code 必须是 str")
    if code != "OK":
        # 错误响应中的 message 不是成功结构的必需字段，所以用 get() 可选读取。
        # 条件表达式 `A if 条件 else B` 根据类型选择服务端消息或本地兜底文本。
        message: Any | None = payload.get("message")
        safe_message: str = message if isinstance(message, str) else "服务端未提供错误说明"
        raise BusinessFailure(code, f"业务码 {code}: {safe_message}")

```

```python
def validate_document_payload(payload: object) -> Document:
        raise BusinessFailure(code, f"业务码 {code}: {safe_message}")

    # 业务码成功后再逐层下钻：payload["data"] -> data["document"]。
    # 每层先验存在/null，再验 dict，避免让 KeyError 或 TypeError 泄漏成含义不清的失败。
    data: object = require_non_null_field(payload, "data", location="响应")
    if not isinstance(data, dict):
        raise ResponseValidationFailure("响应.data 必须是 object/dict")
    document: object = require_non_null_field(data, "document", location="响应.data")
    if not isinstance(document, dict):
        raise ResponseValidationFailure("响应.data.document 必须是 object/dict")

    # 到这里 document 才能安全按键访问。三个局部变量仍是 object，下面的类型判断会把它们
    # 收窄成构造 Document 所需的 int/str/str。
    document_id: object = require_non_null_field(document, "id", location="响应.data.document")
    title: object = require_non_null_field(document, "title", location="响应.data.document")
    content: object = require_non_null_field(document, "content", location="响应.data.document")

    # Python 的 bool 是 int 的子类，isinstance(True, int) 会得到 True；本合同要求真正的
    # 整数 ID，所以使用 type(...) is int 排除布尔值。其余字段允许 str 的子类，因此用 isinstance。
    if type(document_id) is not int:
        raise ResponseValidationFailure("响应.data.document.id 必须是 int，且不能是 bool")
    if not isinstance(title, str):
        raise ResponseValidationFailure("响应.data.document.title 必须是 str")
    if not isinstance(content, str):
        raise ResponseValidationFailure("响应.data.document.content 必须是 str")

    # 关键字参数按名字绑定到 dataclass 字段。这里返回新的稳定对象，而不把仍含 code/data
    # 包装层的原始响应字典泄漏给调用者。
    return Document(id=document_id, title=title, content=content)


def fetch_document(
    query: str,
    *,
    base_url: str | None = None,
    api_token: str | None = None,
```

```python
def fetch_document(
    query: str,
    *,
    base_url: str | None = None,
    api_token: str | None = None,
    session: requests.Session | None = None,
    timeout: tuple[float, float] = DEFAULT_TIMEOUT,
    trace: TraceCallback | None = None,
) -> Document:
    """同步查询一份文档；失败时抛出带层次的异常，不伪造空字典。

    ``base_url`` 与 ``api_token`` 的显式参数优先；未传时才读取环境变量。调用者可以传入
    Session 复用连接，也可以省略它，让函数创建并在本次调用结束时关闭自己的 Session。
    """
    # 签名中 query 后的 * 表示后续参数都必须按名字传入，能减少多个 str/timeout 参数错位。
    # `T | None` 表示值可为 T 或 None；tuple[float, float] 表示正好两个浮点数的类型意图。
    # DEFAULT_TIMEOUT 是不可变 tuple，作为默认值不会产生“多个调用共享可变列表”的陷阱。

    # `or` 具有短路行为：query 不是 str 时不会继续调用不存在的 strip()。
    # strip() 返回去掉首尾空白的新字符串，不会原地修改调用者传入的 str。
    if not isinstance(query, str) or not query.strip():
        raise ValueError("query 必须是非空 str")
    # HTTP 请求尚未发生，先拒绝形状不对或非正数的超时配置。
    if len(timeout) != 2 or timeout[0] <= 0 or timeout[1] <= 0:
        raise ValueError("timeout 必须是两个正数：(connect timeout, read timeout)")

    # base_url 使用 `显式值 or 环境值`：显式传空字符串也会回退到环境变量。
    # token 则用 is not None 区分“没有传”与“明确传入空字符串”；明确的空字符串表示不认证，
    # 不应又从 DOCUMENT_API_TOKEN 捞回一个进程级凭据。
    resolved_base_url: str | None = base_url or os.environ.get("DOCUMENT_API_BASE_URL")
    resolved_token: str | None = api_token if api_token is not None else os.environ.get("DOCUMENT_API_TOKEN")
    if not resolved_base_url:
        raise ConfigurationError("请通过 base_url 参数或 DOCUMENT_API_BASE_URL 配置服务地址")

    # rstrip('/') 只去掉末尾斜杠，再统一拼出 /documents，避免得到双斜杠。
    # params 保持为 Python dict，稍后交给 Requests 做 URL percent-encoding；不要手拼查询字符串。
```

```
189  def fetch_document(
225      url: str = f"{resolved_base_url.rstrip('/')}/documents"
226      params: dict[str, str] = {"query": query.strip()}
227      # Accept 描述希望收到 JSON；GET 没有 JSON body，所以这里不需要请求 Content-Type。
228      headers: dict[str, str] = {"Accept": "application/json"}
229      if resolved_token:
230          headers["Authorization"] = f"Bearer {resolved_token}"
231
232      # 回调是可选依赖；`is not None` 明确区分“没有观察器”和“传入了可调用对象”。
233      if trace is not None:
234          # dict(headers) 建立浅拷贝。后面对 Authorization 的替换只影响日志副本，
235          # 真正发送用的 headers 仍保留完整凭据。
236          safe_headers: dict[str, str] = dict(headers)
237          if "Authorization" in safe_headers:
238              safe_headers["Authorization"] = f"Bearer {mask_secret(resolved_token)}"
239          # trace 接到的是发送前的 Python 数据结构；此刻 Requests 还没有编码 URL 或发出请求。
240          trace(
241              "发送前（Python 对象）",
242              {
243                  "method": "GET",
244                  "url": url,
245                  "params": params,
246                  "headers": safe_headers,
247                  "timeout": {"connect": timeout[0], "read": timeout[1]},
248              },
249          )
250
251      # 调用者传 Session 时复用它的连接池，并继续由调用者拥有；否则本函数创建私有 Session。
252      # 两个变量分开保存客户端与所有权，finally 才能只关闭自己创建的资源。
253      client: Session = session if session is not None else requests.Session()
254      owns_session: bool = session is None
255      try:
256          # 内层 try 只包围真正的网络发送，负责把 Requests 异常翻译成应用异常；外层 try/finally
257          # 则负责 Session 生命周期。client.get(...) 返回后，才拥有可检查的 Response。
258          try:
259              response: Response = client.get(url, params=params, headers=headers, timeout=timeout)
260          except requests.Timeout as error:
```

```python
def fetch_document(
    try:
        # 内层 try 只包围真正的网络发送，负责把 Requests 异常翻译成应用异常；外层 try/finally
        # 则负责 Session 生命周期。client.get(...) 返回后，才拥有可检查的 Response。
        try:
            response: Response = client.get(url, params=params, headers=headers, timeout=timeout)
        except requests.Timeout as error:
            # except 按从上到下匹配；ConnectTimeout 同时也是 ConnectionError，因此必须先捕获
            # Timeout 才不会被归错类。`raise ... from error` 把底层异常保存在 __cause__ 中。
            raise RequestTimeout(f"请求超时 ({type(error).__name__})") from error
        except requests.ConnectionError as error:
            raise ConnectionFailure(f"连接失败 ({type(error).__name__})") from error
        except requests.RequestException as error:
            # RequestException 是 Requests 请求异常的公共基类，接住前两类以外的传输失败，
            # 但不会吞掉普通编程错误或下面的数据校验异常。
            raise TransportError(f"requests 发送请求失败 ({type(error).__name__})") from error

        if trace is not None:
            # 这里已经收到响应，但尚未宣布 HTTP 成功。content 是 bytes，text 是 Requests
            # 按响应编码解码得到的 str；读取这些属性不等于完成 JSON 或业务校验。
            trace(
                "收到后（还未断言成功）",
                {
                    "status_code": response.status_code,
                    "content_type": response.headers.get("Content-Type"),
                    "content_python_type": type(response.content).__name__,
                    "text_python_type": type(response.text).__name__,
                    "body_byte_count": len(response.content),
                },
            )

        # raise_for_status() 对 4xx/5xx 抛 HTTPError，成功状态返回 None；先做这一层，
        # 避免把 404 的合法 JSON 错误正文误送入成功业务模型。
        try:
            response.raise_for_status()
        except requests.HTTPError as error:
```

```
189  def fetch_document(
284
285          # raise_for_status() 对 4xx/5xx 抛 HTTPError，成功状态返回 None；先做这一层，
286          # 避免把 404 的合法 JSON 错误正文误送入成功业务模型。
287          try:
288              response.raise_for_status()
289          except requests.HTTPError as error:
290              # [:120] 产生正文前 120 个字符的新 str；这里只截取本地演示正文，生产日志
291              # 还应按服务的数据分类规则做脱敏。!r 让换行等不可见字符以转义形式展示。
292              preview: str = response.text[:120]
293              raise HttpStatusFailure(
294                  response.status_code,
295                  f"HTTP {response.status_code}，响应预览：{preview!r}",
296              ) from error
297
298          # raise_for_status() 只拒绝 4xx/5xx；3xx、204 等不一定抛错。这个查询合同只允许 200，
299          # 所以还要显式检查精确状态，不能把“未抛 HTTPError”直接等同于业务成功。
300          if response.status_code != 200:
301              raise HttpStatusFailure(
302                  response.status_code,
303                  f"文档查询预期 HTTP 200，实际为 {response.status_code}",
304              )
305
306          # .json() 才把响应正文解析为 Python 对象；它不保证顶层一定是 dict，也不校验字段。
307          try:
308              payload: Any = response.json()
309          except requests.JSONDecodeError as error:
310              # 只把 JSON 语法解析失败转换为本层异常，并保留原始 cause。
311              raise ResponseJsonFailure("HTTP 成功，但响应正文不是合法 JSON") from error
312
313          if trace is not None:
314              # 此处打印的是“可解析但仍不可信”的 Python 值，下一步才做结构和业务规则校验。
315              trace(
316                  "解析后（尚未业务校验）",
317                  {"python_type": type(payload).__name__, "value": payload},
318              )
319
```

```python
def fetch_document(
        if trace is not None:
            # 此处打印的是“可解析但仍不可信”的 Python 值，下一步才做结构和业务规则校验。
            trace(
                "解析后（尚未业务校验）",
                {"python_type": type(payload).__name__, "value": payload},
            )

        # 只有这一调用返回后，payload 才被收窄成调用者可以依赖的 Document。
        document: Document = validate_document_payload(payload)
        if trace is not None:
            trace("校验后（稳定返回值）", document)
        return document
    finally:
        # finally 无论正常 return 还是中途 raise 都执行。仅当本函数创建 Session 时关闭它；
        # 传入的共享 Session 仍由外层调用者继续使用和最终关闭。
        if owns_session:
            client.close()

                I

def post_echo(
    session: requests.Session,
    base_url: str,
    *,
    params: dict[str, str] | None = None,
    json_value: dict[str, str | int] | str | None = None,
    data_value: dict[str, str | int] | str | None = None,
) -> tuple[requests.Response, dict[str, object]]:
    """调用无副作用回显端点，让不同参数生成的真实 HTTP 请求可观察。"""
    # `dict[...] | str | None` 表示示例允许传 Python 字典、已经编码的文本或不传该参数。
    # 函数返回二元 tuple：原始 Response 用来观察最终 URL，dict 用来观察服务端收到的内容。
    # params/json/data 被原样交给 Requests，它们分别控制查询串与两种 body 编码入口。
    response: Response = session.post(
        f"{base_url}/echo",
        params=params,
        json=json_value,
```

```python
def post_echo(
    session: requests.Session,
    base_url: str,
    *,
    params: dict[str, str] | None = None,
    json_value: dict[str, str | int] | str | None = None,
    data_value: dict[str, str | int] | str | None = None,
) -> tuple[requests.Response, dict[str, object]]:
    """调用无副作用回显端点，让不同参数生成的真实 HTTP 请求可观察。"""
    # `dict[...] | str | None` 表示示例允许传 Python 字典、已经编码的文本或不传该参数。
    # 函数返回二元 tuple：原始 Response 用来观察最终 URL，dict 用来观察服务端收到的内容。
    # params/json/data 被原样交给 Requests，它们分别控制查询串与两种 body 编码入口。
    response: Response = session.post(
        f"{base_url}/echo",
        params=params,
        json=json_value,
        data=data_value,
        timeout=DEFAULT_TIMEOUT,
    )
    # 回显端点也先检查 HTTP，再解析 JSON；不能因为它是教学服务就颠倒边界顺序。
    response.raise_for_status()
    payload: Any = response.json()
    # JSON 顶层理论上可以是任意值，本地 fixture 合同则明确要求 object/dict。
    if not isinstance(payload, dict):
        raise TypeError("fixture 的 echo 响应应为 JSON object")
    return response, payload


def demonstrate_json_shapes() -> None:
    """观察 dict、JSON 文本、UTF-8 bytes 和重复序列化。"""
    print_section("1. dict、JSON str 与 UTF-8 bytes")
    # 起点是可按键访问的 Python dict；它只是进程内对象，还不是线上传输的 JSON。
    python_value: dict[str, Any] = {"query": "Agent 入门", "limit": 2}
    # dumps 的 s 表示 string：返回 JSON str。ensure_ascii=False 让中文保留原字符，
    # 随后的 encode 才按 UTF-8 把字符变成可传输的 bytes。
    json_text: str = json.dumps(python_value, ensure_ascii=False)
```

```python
def demonstrate_json_shapes() -> None:
    """观察 dict、JSON 文本、UTF-8 bytes 和重复序列化。"""
    print_section("1. dict、JSON str 与 UTF-8 bytes")
    # 起点是可按键访问的 Python dict；它只是进程内对象，还不是线上传输的 JSON。
    python_value: dict[str, Any] = {"query": "Agent 入门", "limit": 2}
    # dumps 的 s 表示 string：返回 JSON str。ensure_ascii=False 让中文保留原字符，
    # 随后的 encode 才按 UTF-8 把字符变成可传输的 bytes。
    json_text: str = json.dumps(python_value, ensure_ascii=False)
    utf8_bytes: bytes = json_text.encode("utf-8")
    # 反例：json_text 已经是 str，再 dumps 会把“这段字符串”编码成 JSON string，
    # 而不是保留原先 JSON object 的形状，因此内部引号会出现反斜杠转义。
    double_dumped_text: str = json.dumps(json_text, ensure_ascii=False)

    # type(...).__name__ 给出类型名；repr(...) 把引号和反斜杠显式显示出来，便于辨认层次。
    print("Python 数据:", type(python_value).__name__, python_value)
    print("JSON 文本:", type(json_text).__name__, repr(json_text))
    print("UTF-8 载荷:", type(utf8_bytes).__name__, utf8_bytes)
    print("重复 dumps 后:", repr(double_dumped_text))
    # 一次 loads 只撤销最外层序列化，所以先得到 str；对该 str 再 loads 才回到 dict。
    print("一次 loads 的类型:", type(json.loads(double_dumped_text)).__name__)
    print("两次 loads 才回到:", type(json.loads(json.loads(double_dumped_text))).__name__)


def demonstrate_request_arguments(session: requests.Session, base_url: str) -> None:
    """比较 params=、json= 和 data= 在服务端真正看到的结果。"""
    print_section("2. params=、json=、data= 的实际区别")
    # 两个变量内容相近但类型不同：python_value 是 dict，json_text 是已编码的 str。
    python_value: dict[str, Any] = {"query": "Agent 入门", "limit": 2}
    json_text: str = json.dumps(python_value, ensure_ascii=False)

    # tuple 返回值按位置解包：json_response 接 Response，json_echo 接回显 dict。
    # params= 被编码到 URL；json=dict 把 dict 序列化进 body，并设置 JSON Content-Type。
    json_response: Response, json_echo: dict[str, object] = post_echo(
        session,
        base_url,
        params={"source": "第 12 章"},
```

```
383  def demonstrate_request_arguments(session: requests.Session, base_url: str) -> None:
392      json_response: Response, json_echo: dict[str, object] = post_echo(
393          session,
394          base_url,
395          params={"source": "第 12 章"},
396          json_value=python_value,
397      )
398      print("params= 编码后的 URL:", json_response.url)
399      print("json=dict 的 Content-Type:", json_echo["content_type"])
400      print("json=dict 的服务端类型/值:", json_echo["json_value_type"], json_echo["json_value"])
401
402      # `_` 只是"此值不打算使用"的命名惯例，并不是特殊关键字。data=dict 会走表单编码，
403      # 服务端看到 application/x-www-form-urlencoded，而不是 JSON object。
404      _: Response, form_echo: dict[str, object] = post_echo(session, base_url, data_value=python_value)
405      print("data=dict 的 Content-Type:", form_echo["content_type"])
406      print("data=dict 的表单值:", form_echo["form_value"])
407
408      # data=str 直接把现成文本作为 body 发送，不会自动声明 application/json；调用者若真要
409      # 使用这种低层写法，还必须按协议自行管理 Content-Type。
410      _: Response, raw_echo: dict[str, object] = post_echo(session, base_url, data_value=json_text)
411      print("data=JSON str 的 Content-Type:", raw_echo["content_type"])
412      print("data=JSON str 的原始 body:", repr(raw_echo["raw_body"]))
413
414      # 教学反例：json= 期望"待序列化的 Python 值"，传入已经 dumps 的 str 会再次序列化。
415      # 服务端解析一次只得到 str，原始 body 也会出现额外引号和转义符。
416      _: Response, wrong_echo: dict[str, object] = post_echo(session, base_url, json_value=json_text)
417      print("教学反例 json=JSON str 的服务端类型:", wrong_echo["json_value_type"])
418      print("教学反例的原始 body:", repr(wrong_echo["raw_body"]))
419
420
421  def demonstrate_response_views(session: requests.Session, base_url: str) -> None:
422      """同一响应的状态、字节、文本和 JSON 视图回答不同问题。"""
423      print_section("3. Response 的五个观察入口")
424      # GET 的查询条件放 params，不手工处理中文转义；返回值是尚未做业务判断的 Response。
425      response: Response = session.get(
426          f"{base_url}/documents",
427          params={"query": "响应视图"},
```

```python
def demonstrate_response_views(session: requests.Session, base_url: str) -> None:
    """同一响应的状态、字节、文本和 JSON 视图回答不同问题。"""
    print_section("3. Response 的五个观察入口")
    # GET 的查询条件放 params，不手工处理中文转义；返回值是尚未做业务判断的 Response。
    response: Response = session.get(
        f"{base_url}/documents",
        params={"query": "响应视图"},
        timeout=DEFAULT_TIMEOUT,
    )
    # 五个入口各回答一个问题：状态码是 int；成功时 raise_for_status() 的返回值是 None；
    # content 是 bytes；text 是解码后的 str；json() 再解析为 Python 值。
    print("status_code:", response.status_code, type(response.status_code).__name__)
    print("raise_for_status() 返回:", response.raise_for_status())
    print("content:", type(response.content).__name__, response.content[:45])
    print("text:", type(response.text).__name__, response.text[:35])
    parsed: Any = response.json()
    # fixture 的成功响应顶层是 dict；sorted(parsed) 迭代并排序它的键，不会排序嵌套值。
    print("json():", type(parsed).__name__, "顶层字段", sorted(parsed))

    # 这个反例故意凭解析 404 的合法 JSON，再检查状态：前一步能得到业务码 NOT_FOUND，
    # 后一步仍会抛 HTTPError，证明"JSON 可解析"和"HTTP 成功"彼此独立。
    error_response: Response = session.get(
        f"{base_url}/documents",
        params={"query": "http-error"},
        timeout=DEFAULT_TIMEOUT,
    )
    parsed_error: Any = error_response.json()
    print("404 正文仍能解析 JSON:", parsed_error["code"])
    try:
        error_response.raise_for_status()
    except requests.HTTPError as error:
        # type(error).__name__ 只打印类名，让受控演示继续运行；生产代码通常还需保留详情。
        print("但 raise_for_status() 拒绝它:", type(error).__name__)


def demonstrate_fetch_and_session(
    session: requests.Session
```

```python
def demonstrate_fetch_and_session(
    session: requests.Session,
    fixture: LocalHttpFixture,
) -> None:
    """运行完整五层查询，并用服务端记录证明连接被 Session 复用。"""
    print_section("4. fetch_document: 从发送到校验")
    # fixture 会累计记录此前实验的请求；先保存当前长度，稍后用切片只检查本函数新增的记录。
    start_index: int = len(fixture.state.records_snapshot())
    # 两次调用都借用同一个 Session。第一次传 trace 展示四个边界，第二次保持安静，
    # 但两次都必须走完相同的状态、JSON、字段和业务校验才能返回 Document。
    first: Document = fetch_document(
        "Agent",
        base_url=fixture.base_url,
        api_token="local-demo-token-1234",
        session=session,
        trace=trace_to_terminal,
    )
    second: Document = fetch_document(
        "MCP",
        base_url=fixture.base_url,
        api_token="local-demo-token-1234",
        session=session,
    )

    # [start_index:] 取快照尾部；列表推导式只收集 GET 记录的客户端端口，等价于一个紧凑的
    # for + if + append。快照和结果都是新 list，不会修改 fixture 内部记录。
    new_records: list[RequestRecord] = fixture.state.records_snapshot()[start_index:]
    document_ports: list[int] = [record.client_port for record in new_records if record.method == "GET"]
    # `and` 从左到右短路：只有长度确实为 2 时才访问下标 0、1，因此不会因空列表越界。
    connection_reused: bool = len(document_ports) == 2 and document_ports[0] == document_ports[1]
    if not connection_reused:
        # AssertionError 表示教学实验的前提没有被观察到，不属于 Document API 的业务失败。
        raise AssertionError(
            f"支持 keep-alive 的本地服务未观察到 Session 连接复用: {document_ports}"
        )
    # dataclass 自动生成可读的 repr；字段也能像普通对象一样用 second.title 访问。
```

```
456 def demonstrate_fetch_and_session(
491     # dataclass 自动生成可读的 repr；字段也能像普通对象一样用 second.title 访问。
492     print("返回值:", first)
493     print("第二次查询:", second.title)
494     print("服务端观察到两次查询复用客户端端口:", document_ports, connection_reused)
495
496
497 def expect_failure(
498     label: str,
499     expected_type: type[DocumentApiError],
500     *,
501     query: str,
502     session: requests.Session,
503     base_url: str,
504 ) -> None:
505     """只捕获该场景预期的 API 异常；其他缺陷继续中断程序。"""
506     # type[DocumentApiError] 表示参数本身是一种异常"类"，不是异常实例。
507     # Python 允许 except 后使用这个参数，因此同一个小函数可以验证多种预期异常。
508     try:
509         fetch_document(query, base_url=base_url, session=session)
510     except expected_type as error:
511         print(f"{label}: {error.layer} -> {type(error).__name__}: {error}")
512     else:
513         # try 的 else 只在没有异常时执行；这里没有失败反而说明反例未命中预期路径。
514         raise AssertionError(f"{label} 没有产生预期的 {expected_type.__name__}")
515
516
517 def demonstrate_failure_layers(
518     session: requests.Session,
519     fixture: LocalHttpFixture,
520 ) -> None:
521     """让每种失败落在唯一、可说明的边界，不用空 dict 掩盖原因。"""
522     print_section("5. 失败发生在哪一层")
523     # 每个场景是三元 tuple：(展示标签，预期异常类，发给 fixture 的查询词)。
524     # 用数据描述场景后，下面一段循环就能统一验证 HTTP、JSON、字段和业务层失败。
525     cases: list[Any] = [
```

```python
def demonstrate_failure_layers(
    session: requests.Session,
    fixture: LocalHttpFixture,
) -> None:
    """让每种失败落在唯一、可说明的边界，不用空 dict 掩盖原因。"""
    print_section("5. 失败发生在哪一层")
    # 每个场景是三元 tuple：(展示标签，预期异常类，发给 fixture 的查询词)。
    # 用数据描述场景后，下面一段循环就能统一验证 HTTP、JSON、字段和业务层失败。
    cases: list[Any] = [
        ("HTTP 404", HttpStatusFailure, "http-error"),
        ("200 但非法 JSON", ResponseJsonFailure, "invalid-json"),
        ("缺字段", RespponseValidationFailure, "missing-field"),
        ("字段为 null", ResponseValidationFailure, "null-field"),
        ("字段类型不符", ResponseValidationFailure, "wrong-type"),
        ("HTTP/JSON 都成功但业务码失败", BusinessFailure, "business-fail"),
    ]
    # for 会把每个三元 tuple 按位置解包到三个名字；expected_type 仍是类对象。
    for label, expected_type, query in cases:
        expect_failure(
            label,
            expected_type,
            query=query,
            session=session,
            base_url=fixture.base_url,
        )

    # refusing_connection_url() 是上下文管理器：进入时 yield 一个刚释放端口的 URL，
    # 离开时完成其清理。请求该地址会落到 ConnectionFailure，而不是依赖公网制造故障。
    with refusing_connection_url() as unavailable_base_url:
        expect_failure(
            "本机拒绝连接",
            ConnectionFailure,
            query="Agent",
            session=session,
            base_url=unavailable_base_url,
        )
```

```
517  def demonstrate_failure_layers(
554      # slow 场景已建立连接并到达服务端，但在很短的 read timeout 内收不到首字节。
555      # 这里单独验证 RequestTimeout，并用 finally 确保任何离开路径都会释放服务端等待事件。
556      try:
557          fetch_document(
558              "slow",
559              base_url=fixture.base_url,
560              session=session,
561              timeout=SLOW_RESPONSE_TIMEOUT,
562          )
563      except RequestTimeout as error:
564          print(f"慢响应: {error.layer} -> {type(error).__name__}: {error}")
565      else:
566          raise AssertionError("慢响应没有产生预期的 RequestTimeout")
567      finally:
568          # 客户端已经完成超时分类，现在明确解除服务端事件；set() 会把事件置为已触发，
569          # 在有人调用 clear() 前保持该状态，正在 wait() 的服务线程因此可以继续收尾。
570          fixture.state.release_slow_response.set()
571
572      # is_set() 证明请求确实到达服务端；wait(timeout=...) 返回 bool，True 表示清理事件按时发生。
573      # 这比固定 sleep 后猜测线程顺序更可靠，也避免反例永久挂住程序。
574      if not fixture.state.slow_request_started.is_set():
575          raise AssertionError("慢请求没有到达服务端，不能把结果解释成 read timeout")
576      if not fixture.state.slow_request_finished.wait(timeout=1.0):
577          raise AssertionError("慢请求服务线程没有清理完成")
578      print("慢请求顺序证据：服务端已进入等待 -> 客户端 read timeout -> 事件释放并清理")
579      print("所有失败都保留异常类型与原因，没有 except Exception 后返回 {}。")
580
581
582  def main() -> None:
583      """默认只访问自动管理的 127.0.0.1 服务，不访问公网、不需要真实 API Key。"""
584      print("Python HTTP/JSON 边界实验：仅访问自动启动的本地 fixture")
585      # 安装版本属于发行包元数据；metadata.version 按发行名 requests 查询当前环境，
586      # 不依赖库是否把 __version__ 作为公开类型接口导出。
587      print("requests 版本:", metadata.version("requests"))
588      # 这一段纯本地转换不需要服务器，因此先运行。
```

```
581
582 def main() -> None:
583     """默认只访问自动管理的 127.0.0.1 服务，不访问公网、不需要真实 API Key。"""
584     print("Python HTTP/JSON 边界实验：仅访问自动启动的本地 fixture")
585     # 安装版本属于发行包元数据；metadata.version 按发行名 requests 查询当前环境，
586     # 不依赖库是否把 __version__ 作为公开类型接口导出。
587     print("requests 版本:", metadata.version("requests"))
588     # 这一段纯本地转换不需要服务器，因此先运行。
589     demonstrate_json_shapes()
590
591     # `with LocalHttpFixture() as fixture` 可类比 Java try-with-resources：进入时启动本地服务，
592     # 无论块内正常结束还是抛异常，离开时都会关闭监听 socket 并等待服务线程结束。
593     with LocalHttpFixture() as fixture:
594         print("本地服务:", fixture.base_url)
595         # Session 自己也是上下文管理器。嵌套 with 明确表示连接池的生命周期短于 fixture，
596         # 因此先关闭客户端连接池，再关闭它所连接的本地服务。
597         with requests.Session() as session:
598             # update 原地修改 Session 的公共 headers；每次调用传入的 headers 会与之合并。
599             session.headers.update({"X-Demo-Client": "chapter-12"})
600             # 本地 fixture 不应受机器上的 HTTP_PROXY / .netrc 等环境配置影响；False 只为保持
601             # 本机实验输入可控，不代表真实项目应无条件禁用组织代理或环境认证。
602             session.trust_env = False
603             # 同一个 Session 顺序执行后续真实 HTTP 实验，允许 urllib3 连接池复用连接。
604             demonstrate_request_arguments(session, fixture.base_url)
605             demonstrate_response_views(session, fixture.base_url)
606             demonstrate_fetch_and_session(session, fixture)
607             demonstrate_failure_layers(session, fixture)
608         print_section("6. 资源关闭")
609         print("已退出 Session 的 with 块；连接池已关闭。")
610
611     print("已退出 fixture 的 with 块；监听端口和服务线程已关闭:", fixture.closed)
612     print("完整示例执行完毕。")
613
614
615 # 导入本文件时，函数和类只会被定义，不会自动跑实验。只有把它作为脚本直接执行时，
616 # Python 才把模块名设为 "__main__" 并调用 main()；这相当于显式声明程序入口。
617 if __name__ == "__main__":
```

```
582  def main() -> None:
600              # 本地 fixture 不应受机器上的 HTTP_PROXY / .netrc 等环境配置影响；False 只为保持
601              # 本机实验输入可控，不代表真实项目应无条件禁用组织代理或环境认证。
602              session.trust_env = False
603              # 同一个 Session 顺序执行后续真实 HTTP 实验，允许 urllib3 连接池复用连接。
604              demonstrate_request_arguments(session, fixture.base_url)
605              demonstrate_response_views(session, fixture.base_url)
606              demonstrate_fetch_and_session(session, fixture)
607              demonstrate_failure_layers(session, fixture)
608          print_section("6. 资源关闭")
609          print("已退出 Session 的 with 块；连接池已关闭。")
610
611      print("已退出 fixture 的 with 块；监听端口和服务线程已关闭:", fixture.closed)
612      print("完整示例执行完毕。")
613
614
615  # 导入本文件时，函数和类只会被定义，不会自动跑实验。只有把它作为脚本直接执行时，
616  # Python 才把模块名设为 "__main__" 并调用 main()；这相当于显式声明程序入口。
617  if __name__ == "__main__":
618      main()
619
```

```python
"""第 12 章的本地 HTTP fixture。

本文件刻意把线程、socket 和 HTTP 服务端细节隔离起来。学习客户端代码时，只需把
``LocalHttpFixture`` 当成一个进入 ``with`` 后可用、离开后自动关闭的本地 API。
"""

import json
import socket
import threading
from contextlib import contextmanager
from dataclasses import dataclass, field
from http import HTTPStatus
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from typing import Self, cast
from urllib.parse import parse_qs, urlsplit


@dataclass(frozen=True)
class RequestRecord:
    """fixture 观察到的一次请求；只保存验证连接复用所需的公开信息。"""

    method: str
    target: str
    client_port: int


@dataclass
class FixtureState:
    """在线程之间共享事件，并以加锁快照暴露请求记录。"""

    slow_request_started: threading.Event = field(default_factory=threading.Event)
    release_slow_response: threading.Event = field(default_factory=threading.Event)
    slow_request_finished: threading.Event = field(default_factory=threading.Event)
    server_thread_started: threading.Event = field(default_factory=threading.Event)
    _records: list[RequestRecord] = field(default_factory=list)
    _records_lock: threading.Lock = field(default_factory=threading.Lock)

```

```python
@dataclass
class FixtureState:
    """在线程之间共享事件，并以加锁快照暴露请求记录。"""

    slow_request_started: threading.Event = field(default_factory=threading.Event)
    release_slow_response: threading.Event = field(default_factory=threading.Event)
    slow_request_finished: threading.Event = field(default_factory=threading.Event)
    server_thread_started: threading.Event = field(default_factory=threading.Event)
    _records: list[RequestRecord] = field(default_factory=list)
    _records_lock: threading.Lock = field(default_factory=threading.Lock)

    def record(self, method: str, target: str, client_port: int) -> None:
        """请求可能由不同服务线程处理，所以写入共享列表时需要锁。"""
        with self._records_lock:
            self._records.append(RequestRecord(method, target, client_port))

    def records_snapshot(self) -> list[RequestRecord]:
        """返回副本，避免调用者在锁外迭代一个正在变化的列表。"""
        with self._records_lock:
            return list(self._records)


class FixtureServer(ThreadingHTTPServer):
    """给标准库服务器附加本章状态；工作线程不会阻止进程退出。"""

    daemon_threads = True

    def __init__(
        self,
        server_address: tuple[str, int],
        handler_class: type[BaseHTTPRequestHandler],
        state: FixtureState,
    ) -> None:
        self.state: FixtureState = state
        super().__init__(server_address, handler_class)

```

```python

class FixtureServer(ThreadingHTTPServer):
    """给标准库服务器附加本章状态；工作线程不会阻止进程退出。"""

    daemon_threads = True

    def __init__(
        self,
        server_address: tuple[str, int],
        handler_class: type[BaseHTTPRequestHandler],
        state: FixtureState,
    ) -> None:
        self.state: FixtureState = state
        super().__init__(server_address, handler_class)


class FixtureRequestHandler(BaseHTTPRequestHandler):
    """提供文档查询和无副作用回显端点；不是本章要求掌握的服务端实现。"""

    # HTTP/1.1 加上准确的 Content-Length，允许 Session 复用同一条本地 TCP 连接。
    protocol_version = "HTTP/1.1"

    @property
    def fixture_server(self) -> FixtureServer:
        if not isinstance(self.server, FixtureServer):
            raise TypeError("fixture handler 必须由 FixtureServer 承载")
        return self.server

    def log_message(self, format_string: str, *args: object) -> None:
        """关闭标准库逐请求日志，避免教学输出被服务线程穿插。"""

    def do_GET(self) -> None:
        """GET /documents 是只读查询，没有修改资源的语义。"""
        self.fixture_server.state.record("GET", self.path, self.client_address[1])
        target: SplitResult = urlsplit(self.path)
        if target.path != "/documents":
```

```
63
64  class FixtureRequestHandler(BaseHTTPRequestHandler):
65      """提供文档查询和无副作用回显端点；不是本章要求掌握的服务端实现。"""
66
67      # HTTP/1.1 加上准确的 Content-Length，允许 Session 复用同一条本地 TCP 连接。
68      protocol_version = "HTTP/1.1"
69
70      @property
71      def fixture_server(self) -> FixtureServer:
72          if not isinstance(self.server, FixtureServer):
73              raise TypeError("fixture handler 必须由 FixtureServer 承载")
74          return self.server
75
76      def log_message(self, format_string: str, *args: object) -> None:
77          """关闭标准库逐请求日志，避免教学输出被服务线程穿插。"""
78
79      def do_GET(self) -> None:
80          """GET /documents 是只读查询，没有修改资源的语义。"""
81          self.fixture_server.state.record("GET", self.path, self.client_address[1])
82          target: SplitResult = urlsplit(self.path)
83          if target.path != "/documents":
84              self._send_json(
85                  HTTPStatus.NOT_FOUND,
86                  {"code": "NOT_FOUND", "message": "本地 fixture 没有这个端点"},
87              )
88              return
89
90          query_values: list[str] = parse_qs(target.query, keep_blank_values=True).get("query", [])
91          query: str = query_values[0] if query_values else ""
92          if not query:
93              self._send_json(
94                  HTTPStatus.BAD_REQUEST,
95                  {"code": "INVALID_QUERY", "message": "query 不能为空"},
96              )
97              return
98
99          if query == "http-error":
```

```
 64  class FixtureRequestHandler(BaseHTTPRequestHandler):
 79      def do_GET(self) -> None:
 99          if query == "http-error":
100              # 错误响应故意也是合法 JSON，用来证明“能解析 JSON”不等于 HTTP 成功。
101              self._send_json(
102                  HTTPStatus.NOT_FOUND,
103                  {"code": "NOT_FOUND", "message": "文档不存在"},
104              )
105              return
106          if query == "invalid-json":
107              self._send_bytes(
108                  HTTPStatus.OK,
109                  b'{"code": "OK", this-is-not-json',
110                  content_type="application/json; charset=utf-8",
111              )
112              return
113          if query == "missing-field":
114              self._send_json(
115                  HTTPStatus.OK,
116                  {
117                      "code": "OK",
118                      "data": {"document": {"id": 12, "content": "缺少 title"}},
119                  },
120              )
121              return
122          if query == "null-field":
123              self._send_json(
124                  HTTPStatus.OK,
125                  {
126                      "code": "OK",
127                      "data": {"document": {"id": 12, "title": None, "content": "title 为 null"}},
128                  },
129              )
130              return
131          if query == "wrong-type":
132              self._send_json(
133                  HTTPStatus.OK
```

```
class FixtureRequestHandler(BaseHTTPRequestHandler):
    def do_GET(self) -> None:
            return
        if query == "wrong-type":
            self._send_json(
                HTTPStatus.OK,
                {
                    "code": "OK",
                    "data": {
                        "document": {
                            "id": "12",
                            "title": "类型错误",
                            "content": "id 应为 int",
                        }
                    },
                },
            )
            return
        if query == "business-fail":
            self._send_json(
                HTTPStatus.OK,
                {"code": "QUOTA_EXCEEDED", "message": "本地演示额度不足", "data": None},
            )
            return
        if query == "slow":
            self._send_slow_document()
            return

        self._send_json(
            HTTPStatus.OK,
            {
                "code": "OK",
                "data": {
                    "document": {
                        "id": 12,
                        "title": f"{query} 查询结果",
                        "content": "一次 API 调用要逐层检查。",
```

```
 64  class FixtureRequestHandler(BaseHTTPRequestHandler):
 79      def do_GET(self) -> None:
155
156          self._send_json(
157              HTTPStatus.OK,
158              {
159                  "code": "OK",
160                  "data": {
161                      "document": {
162                          "id": 12,
163                          "title": f"{query} 查询结果",
164                          "content": "一次 API 调用要逐层检查。",
165                      }
166                  },
167              },
168          )
169
170      def do_POST(self) -> None:
171          """POST /echo 只回显收到的数据，不保存、创建或修改业务资源。"""
172          self.fixture_server.state.record("POST", self.path, self.client_address[1])
173          target: SplitResult = urlsplit(self.path)
174          if target.path != "/echo":
175              self._send_json(
176                  HTTPStatus.NOT_FOUND,
177                  {"code": "NOT_FOUND", "message": "本地 fixture 没有这个端点"},
178              )
179              return
180
181          content_length = int(self.headers.get("Content-Length", "0"))
182          body_bytes: bytes = self.rfile.read(content_length)
183          content_type: str = self.headers.get("Content-Type", "")
184          body_text: str = body_bytes.decode("utf-8", errors="replace")
185
186          json_value: object | None = None
187          json_value_type: str | None = None
188          form_value: dict[str, list[str]] | None = None
189          if content_type.startswith("application/json"):
```

```
 64  class FixtureRequestHandler(BaseHTTPRequestHandler):
170      def do_POST(self) -> None:
188          form_value: dict[str, list[str]] | None = None
189          if content_type.startswith("application/json"):
190              try:
191                  json_value = json.loads(body_text)
192              except json.JSONDecodeError:
193                  json_value_type = "解析失败"
194              else:
195                  json_value_type = type(json_value).__name__
196          elif content_type.startswith("application/x-www-form-urlencoded"):
197              form_value = parse_qs(body_text, keep_blank_values=True)
198
199          self._send_json(
200              HTTPStatus.OK,
201              {
202                  "method": "POST",
203                  "query_params": parse_qs(target.query, keep_blank_values=True),
204                  "content_type": content_type or None,
205                  "raw_body": body_text,
206                  "json_value": json_value,
207                  "json_value_type": json_value_type,
208                  "form_value": form_value,
209                  "demo_header": self.headers.get("X-Demo-Client"),
210              },
211          )
212
213      def _send_slow_document(self) -> None:
214          """用事件阻塞首字节，直到客户端超时后明确释放，不用 sleep 猜线程顺序。"""
215          state: FixtureState = self.fixture_server.state
216          state.slow_request_started.set()
217          # 两秒只是 fixture 自我清理的上限；正常示例由客户端 finally 立即释放这个事件。
218          released_by_client: bool = state.release_slow_response.wait(timeout=2.0)
219          try:
220              if released_by_client:
221                  self._send_json(
222                      HTTPStatus.OK,
```

```
 64  class FixtureRequestHandler(BaseHTTPRequestHandler):
212
213      def _send_slow_document(self) -> None:
214          """用事件阻塞首字节，直到客户端超时后明确释放，不用 sleep 猜线程顺序。"""
215          state: FixtureState = self.fixture_server.state
216          state.slow_request_started.set()
217          # 两秒只是 fixture 自我清理的上限；正常示例由客户端 finally 立即释放这个事件。
218          released_by_client: bool = state.release_slow_response.wait(timeout=2.0)
219          try:
220              if released_by_client:
221                  self._send_json(
222                      HTTPStatus.OK,
223                      {
224                          "code": "OK",
225                          "data": {
226                              "document": {
227                                  "id": 12,
228                                  "title": "慢响应",
229                                  "content": "客户端已经不再等待这份响应。",
230                              }
231                          },
232                      },
233                  )
234          finally:
235              state.slow_request_finished.set()
236
237      def _send_json(self, status: HTTPStatus, payload: object) -> None:
238          """统一用 UTF-8 编码 JSON，并让中文在抓包/输出中保持可读。"""
239          body: bytes = json.dumps(payload, ensure_ascii=False).encode("utf-8")
240          self._send_bytes(status, body, content_type="application/json; charset=utf-8")
241
242      def _send_bytes(self, status: HTTPStatus, body: bytes, *, content_type: str) -> None:
243          """总是发送长度，且把超时后客户端断开视为 fixture 的预期路径。"""
244          self.send_response(status)
245          self.send_header("Content-Type", content_type)
246          self.send_header("Content-Length", str(len(body)))
247          self.end_headers()
```

```python
class FixtureRequestHandler(BaseHTTPRequestHandler):

    def _send_bytes(self, status: HTTPStatus, body: bytes, *, content_type: str) -> None:
        """总是发送长度，且把超时后客户端断开视为 fixture 的预期路径。"""
        self.send_response(status)
        self.send_header("Content-Type", content_type)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        try:
            self.wfile.write(body)
        except BrokenPipeError, ConnectionResetError:
            # 慢响应实验中，客户端读超时会先关闭连接；服务端稍后写入失败是预期结果。
            return


class LocalHttpFixture:
    """自动选择本地端口，并负责启动、关闭和等待服务线程。"""

    def __init__(self) -> None:
        self.state = FixtureState()
        self._server: FixtureServer | None = None
        self._thread: threading.Thread | None = None
        self.base_url: Literal[''] = ""
        self.closed = False

    def __enter__(self) -> Self:
        server = FixtureServer(("127.0.0.1", 0), FixtureRequestHandler, self.state)
        self._server = server
        # FixtureServer 固定 AF_INET；cast 排除 typeshed 仍保留的 IPv6 四元组可能性。
        host: str, port: int = cast(tuple[str, int], server.server_address)
        self.base_url: str = f"http://{host}:{port}"

        def serve() -> None:
            self.state.server_thread_started.set()
            server.serve_forever(poll_interval=0.02)

        thread = threading.Thread(target=serve, name="chapter12-http-fixture", daemon=True)
```

```
255  class LocalHttpFixture:
265      def __enter__(self) -> Self:
275
276          thread = threading.Thread(target=serve, name="chapter12-http-fixture", daemon=True)
277          self._thread = thread
278          thread.start()
279          if not self.state.server_thread_started.wait(timeout=1.0):
280              self.__exit__(None, None, None)
281              raise RuntimeError("本地 HTTP fixture 未能启动")
282          return self
283
284      def __exit__(self, exc_type: object, exc_value: object, traceback: object) -> None:
285          # 无论示例从哪条异常路径离开，都先解除慢请求，再关闭监听端口。
286          self.state.release_slow_response.set()
287          if self._server is not None:
288              self._server.shutdown()
289              self._server.server_close()
290          if self._thread is not None:
291              self._thread.join(timeout=2.0)
292              if self._thread.is_alive():
293                  raise RuntimeError("本地 HTTP fixture 服务线程没有按期退出")
294          self.closed = True
295
296
297  @contextmanager
298  def refusing_connection_url() -> Generator[str, Any, None]:
299      """找出一个刚释放的本地端口，用来制造拒绝连接并观察 ConnectionError。"""
300      # 先让内核选择端口，再在发起请求前关闭 socket。若保持“已绑定但未监听”，部分系统会让
301      # connect 等到超时；释放后本机 TCP 栈会立即拒绝连接，更符合这个实验要观察的错误。
302      with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as probe:
303          probe.bind(("127.0.0.1", 0))
304          host: _RetAddress, port: _RetAddress = probe.getsockname()
305      yield f"http://{host}:{port}"
306
```

```
1  requests==2.34.2
2
```